

浙江大学

本科实验报告

课程名称：计算机组成与设计

姓名：张序鸿

学院：竺可桢学院

专业：计算机科学与技术

学号：3230103265

指导教师：杨莹春

2024 年 11 月 15 日

浙江大学实验报告

课程名称: 计算机组成与设计 实验类型: 综合

实验项目名称: CPU 设计之数据通路

学生姓名: 张序鸿 专业: 混合班 学号: 3230103265

指导老师: 杨莹春、潘之杰 助教: 张立冬、马致远

实验地点: 东 4-509 实验日期: 2024 年 11 月 20 日

一、实验目的和要求

1. 复习寄存器传输控制技术
2. 掌握 CPU 的核心组成: 数据通路与控制器
3. 掌握 CPU 的核心: 数据通路组成与原理, 并设计 Datapath
4. 学习测试方案的设计

二、实验内容和原理

内容:

任务一: 设计实现数据通路 —— 采用 RTL 实现

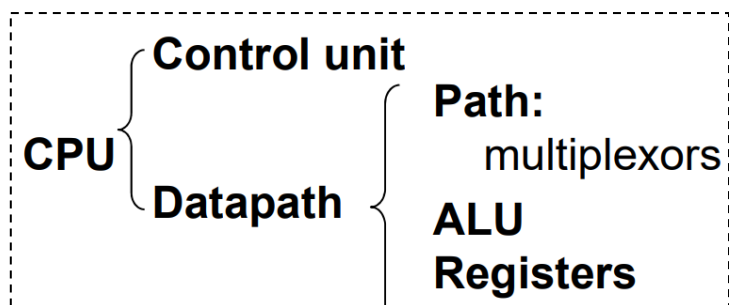
- ALU 和 Regs 调用 Exp01 设计的模块
- PC 寄存器设计及 PC 通路建立 π
- ImmGen 立即数生成模块设计
- 此实验在 Exp4-0 的基础上完成, 替换 Exp4-0 的数据通路核

任务二: 设计数据通路测试方案并完成测试

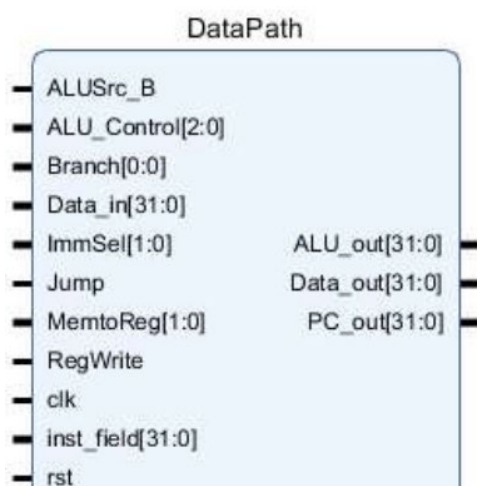
- 通路测试: I-格式通路、R-格式通路
- 部件测试: ALU、Register Files

原理：

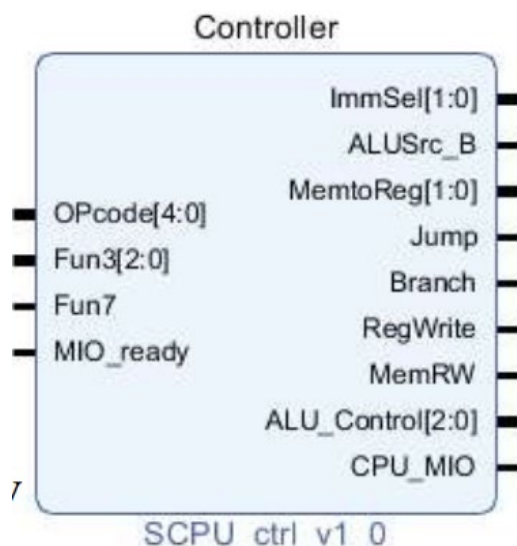
2.1 计算机系统中 CPU 组成



- 数据通路 Data_path 作为 CPU 主要部件之一，具有通用计算功能的算术逻辑部件（主要为 ALU）、具有通用目的寄存器堆、具有通用计数所需的尽可能的路径。



- 控制器 SCPU_ctrl 是寄存器传输控制技术中的运算和通路控制器，它起到指令译码（inst_in 的分块）、产生操作控制信号（ALU 运算控制）、产生指令所需的路径选择的作用。

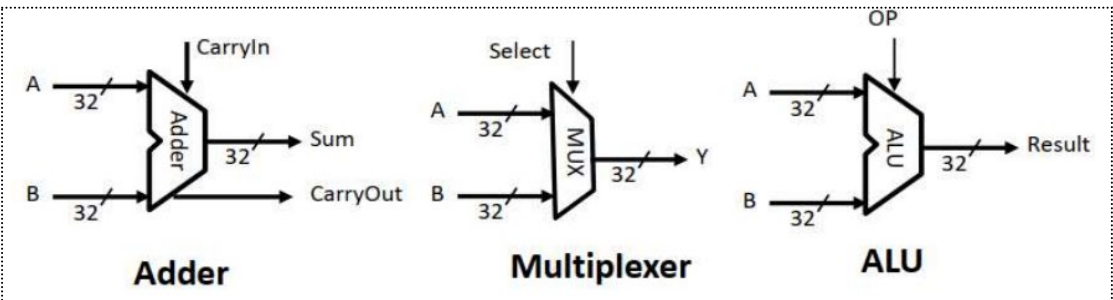


2.2 RISC-V 数据通路 Datapath 原理介绍

2.2.1 数据通路组成部分

包含了完成处理器所要求的的操作所必须的硬件，包括运算单元、寄存器组、状态寄存器等

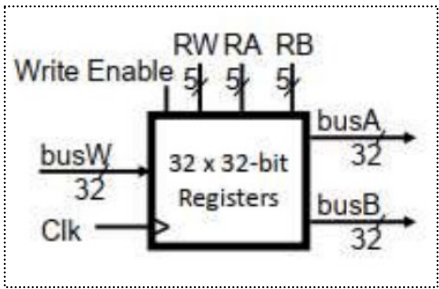
(1) 组合逻辑单元：加法器、多路选择器、ALU 运算单元、立即数生成模块 ImmGen



Instruction type	Instruction opcode[6:0]	Instruction operation	(sign-extend)immediate	Imm Sel
I-type	0000011	Lw;lb;lh;lb;lh	(sign-extend) instr[31:20]	00
	0010011	Addi;slti;sltiu;xori;ori;andi;		
	1100111	jair		
S-type	0100011	Sw;sb;sh	(sign-extend) instr[31:25],[11:7]	01
B-type	1100011	Beq;bne;blt;bge;bltu;bgeu	(sign-extend) instr[31],[7],[30:25],[11:8],1'b0	10
J-type	1101111	jal	(sign-extend) instr[31],[19:12],[20],[30:21],1'b0	11
2024/3/21			Chapter 6	19

ImmGen 利用后续 SCPU_ctrl 给出的 ImmSel 为不同格式的指令生成 32 位立即数（前二十位是符号位拓展，见上图）。

(2) 时序逻辑单元：寄存器堆（Register Files）、存储器（Data Memory）



Register Files 由 32 个 register 组成，包括 1 个 32 位数据输入端口（RegWrite）、2 个 32 位数据输出端口（Rs1_data、Rs2_data）。

(3) 寄存器类型分组：

Register(s)	Alt.	Description
x0	zero	The zero register, always zero
x1	ra	The return address register, stores where functions should return
x2	sp	The stack pointer, where the stack ends
x5-x7, x28-x31	t0-t6	The temporary registers
x8-x9, x18-x27	s0-s11	The saved registers
x10-x17	a0-a7	The argument registers, a0-a1 are also return value

2.2.2 RISC-V 指令

(1) RISC-V 基本指令格式：

寄存器-寄存器操作的 R 型指令；短立即数和访存 load 操作的 I 型指令；访存 store 操作的 S 型指令；条件跳转操作的 B 类型指令；长立即数的 U 型指令和无条件跳转的 J 型指令。

如下图给出了各个指令的编码规则：

31	30	25	24	21	20	19	15	14	12	11	8	7	6	0			
funct7				rs2			rs1		funct3		rd			opcode		R-type	
imm[11:0]						rs1		funct3		rd			opcode		I-type		
imm[11:5]				rs2			rs1		funct3		imm[4:0]			opcode		S-type	
imm[12]		imm[10:5]			rs2			rs1		funct3		imm[4:1]		imm[11]		opcode	B-type
imm[31:12]										rd			opcode		U-type		
imm[20]		imm[10:1]			imm[11]			imm[19:12]			rd			opcode		J-type	

- **opcode**：操作码，用于表示指令格式和指令操作的字段
- **rs1**：只读。第 1 个源操作数寄存器，寄存器地址（编号）是 0x00~0x1F；
- **rs2**：只读。第 2 个源操作数寄存器，寄存器地址（编号）是 0x00~0x1F；
- **rd**：只写。目的操作数寄存器，寄存器地址（编号）是 0x00~0x1F；
- **funct3/7**：功能码，在指令中用来指定指令的功能与操作码配合使用；
- **immediate**：立即数，用作无符号的逻辑操作数、有符号的算术操作数、数据加载（Load）/数据保存（Store）指令的数据地址字节偏移量和分支指令中相对程序计数器（PC）的有符号偏移量；

【对于同一指令格式中的指令来说，数据通路、opcode 基本相同并通用，主要通过 funct3/7 来进行功能决定】

(2) 本 Datapath 实现的指令：

R : add , sub , and , or , xor , srl , slt ;

I : addi , andi , ori , xori , srli , slti , lw ;

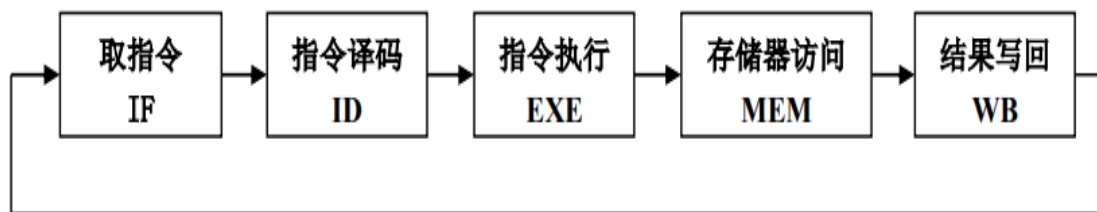
S : sw ; **B** : beq ; **J** : jal

Lab4-03 中需要拓展以上 5 类指令，并添加 U 型指令。

2.3 单周期 CPU 简单介绍

单周期 CPU 指的是一条指令的执行在一个时钟周期内完成，然后开始下一条指令的执行，即一条指令用一个时钟周期完成。

我们讨论的单周期主要包括以下五个阶段：



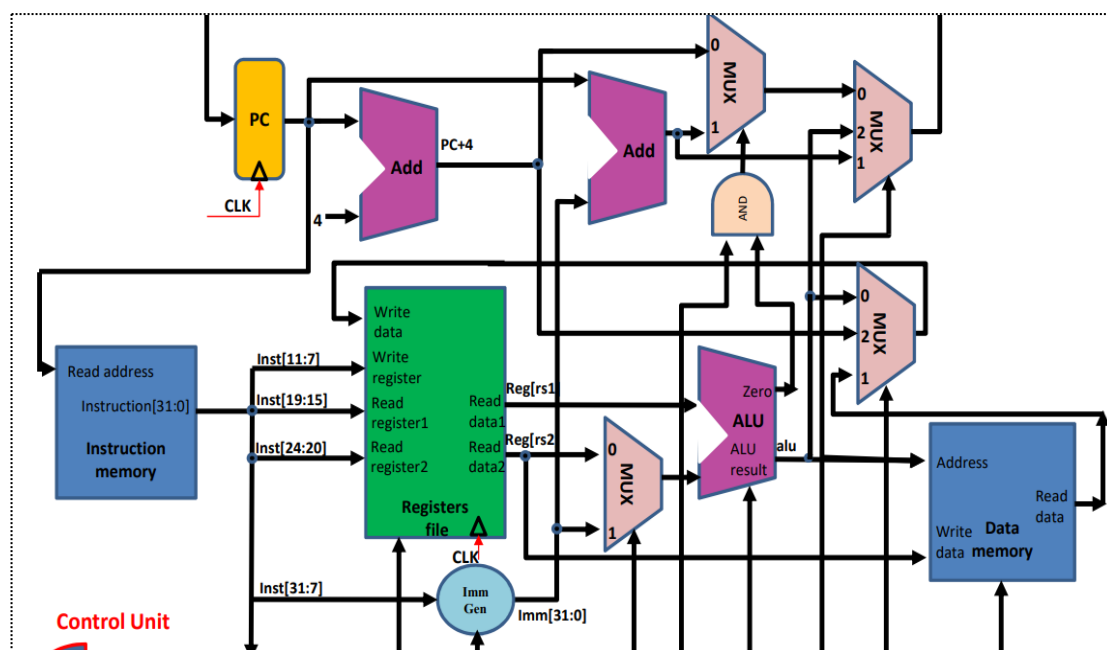
(1) **取指令(IF)**：根据程序计数器 PC 中的指令地址，从存储器中取出一条指令，同时，PC 根据指令字长度自动递增产生下一条指令所需要的指令地址，但遇到“地址转移”指令时，则控制器把“转移地址”送入 PC。

(2) **指令译码(ID)**：对取指令操作中得到的指令进行分析并译码，确定这条指令需要完成的操作，从而产生相应的操作控制信号，用于驱动执行状态中的各种操作。

(3) **指令执行(EXE)**：根据指令译码得到的操作控制信号，具体地执行指令动作，然后转移到结果写回状态。

(4) **存储器访问(MEM)**：所有需要访问存储器的操作都将在这个步骤中执行，该步骤给出存储器的数据地址，把数据写入到存储器中数据地址所指定的存储单元或者从存储器中得到数据地址单元中的数据。

(5) **结果写回(WB)**：指令执行的结果或者访问存储器中得到的数据写回相应的目的寄存器中。



【单周期 CPU 数据通路结构】

- Instruction Memory: 指令存储器,
- Registers file: 寄存器组
- ALU: 算术逻辑单元 (zero 常用于跳转指令)
- Data Memory: 数据存储器,
- PC: 程序计数器 (存放指令地址)
- ImmGen : 立即数生成单元

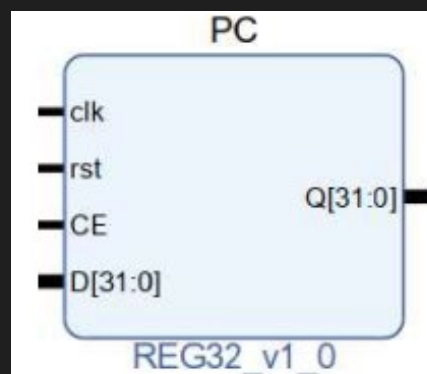
2.4 部分新增模块介绍

(1) PC 寄存器 (REG32.v):

用于指令地址的锁存, 采用行为描述设计。

```

module REG32(
    input clk, // 上升沿触发
    input rst,
    input CE,
    input [31:0]D,
    output reg [31:0]Q);
    always @(posedge clk or posedge rst)
        if (rst==1)
            Q <= 32'b0;
        else if (CE == 1)
            Q <= D;
endmodule
    
```

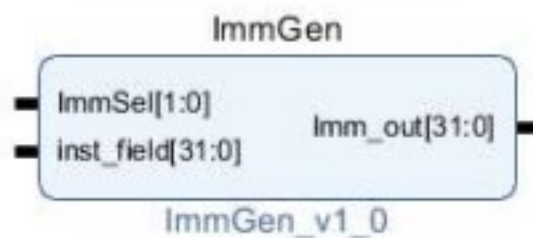


(2) ImmGen 立即数生成模块:

根据输入指令产生立即数的逻辑功能，以及转移指令偏移量左移位的功能。

```
`timescale 1ns / 1ps
module ImmGen(
    input wire [1:0] ImmSel, // 立即数操作控制
    input wire [31:0] inst_field, // Imm 指令数据范围【31: 7】
    output reg [31:0] Imm_out // 立即数输出
);

always@*begin
    case(ImmSel)
2'b00:Imm_out = {{20{inst_field[31]}},inst_field[31:20]}; //addi\lw(I)
2'b01:Imm_out = {{20{inst_field[31]}},inst_field[31:25],inst_field[11:7]}; //sw(s)
2'b10:Imm_out =
{{20{inst_field[31]}},inst_field[7],inst_field[30:25],inst_field[11:8],1'b0};
//beq(b)
2'b11:Imm_out =
{{12{inst_field[31]}},inst_field[19:12],inst_field[20],inst_field[30:21],1'b0};
//jal(j)
    endcase
    end
endmodule
```



三、实验过程和数据记录

3.1 设计实现 Datapath:

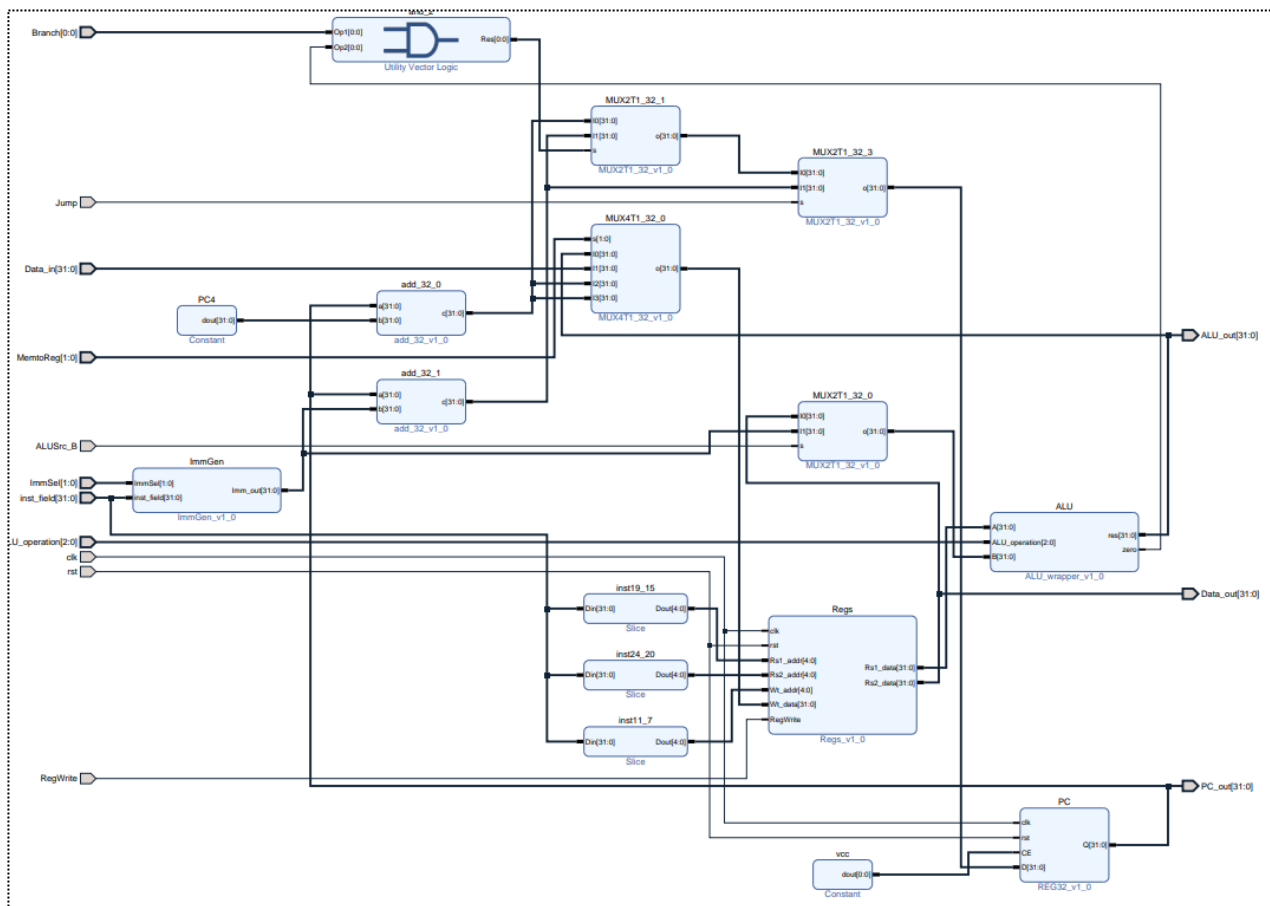
(1) Vivado 新建工程

- 利用 Vivado 新建工程，命名 OExp04-Datapath ；
- 最后正确选择 FPGA 芯片——xc7a100tcsg324-1；

本实验需要用到 Lab0 设计的 MUX 多路选择器、若干基本运算模块(and、or、ADC、xor、nor、srl 等)，以及 Lab1 设计的 ALU、Regs。

同时需要创建 PC、ImmGen 两个模块（具体代码见实验原理）。

(2) 设计 CPU 中的 Datapath:



逻辑原理图如上，代码实现如下：

```
`timescale 1ns / 1ps

module DataPath(
    input wire clk,
    input wire rst,
    input wire[31:0] inst_field,
    input wire[31:0] Data_in,
    input wire[2:0] ALU_operation,
    input wire[1:0] ImmSel,
    input wire[1:0] MemtoReg,
    input wire ALUSrc_B,
    input wire Jump,
    input wire Branch,
    input wire RegWrite,
    output wire[31:0] PC_out,
    output wire[31:0] Data_out,
    output wire[31:0] ALU_out
);

    wire[31:0] Imm_out;
    wire[31:0] PC_inc;
    wire[31:0] PC_imm;
    wire ALU_zero;
    wire[31:0] PC_1;
    wire[31:0] Wt_data;
    wire[31:0] PC_new;
    wire[31:0] Rs1_data;
    wire[31:0] ALU_B;
    ImmGen ImmGen(
        .ImmSel(ImmSel),
        .inst_field(inst_field),
        .Imm_out(Imm_out)
    );
    add32 add_32_0(
        .a(PC_out),
        .b(32'h4),
        .c(PC_inc)
    );
    add32 add_32_1(
        .a(PC_out),
        .b(Imm_out),
        .c(PC_imm)
    );
);
```

```

MUX2T1_32 MUX2T1_32_1(
    .I0(PC_inc),
    .I1(PC_imm),
    .s(Branch & ALU_zero),
    .o(PC_1)
);
MUX4T1_32 MUX4T1_32_0(
    .s(MemtoReg),
    .I0(ALU_out),
    .I1(Data_in),
    .I2(PC_inc),
    .I3(PC_inc),
    .o(Wt_data)
);
MUX2T1_32 MUX2T1_32_3(
    .I0(PC_1),
    .I1(PC_imm),
    .s(Jump),
    .o(PC_new)
);
MUX2T1_32 MUX2T1_32_0(
    .I0(Data_out),
    .I1(Imm_out),
    .s(ALUSrc_B),
    .o(ALU_B)
);
Regs Regs(
    .clk(clk),
    .rst(rst),
    .RegWrite(RegWrite),
    .Rs1_addr(inst_field[19:15]),
    .Rs2_addr(inst_field[24:20]),
    .Wt_addr(inst_field[11:7]),
    .Wt_data(Wt_data),
    .Rs1_data(Rs1_data),
    .Rs2_data(Data_out)
);
ALU ALU(
    .A(Rs1_data),
    .B(ALU_B),
    .ALU_operation(ALU_operation),
    .res(ALU_out),
    .zero(ALU_zero)
);

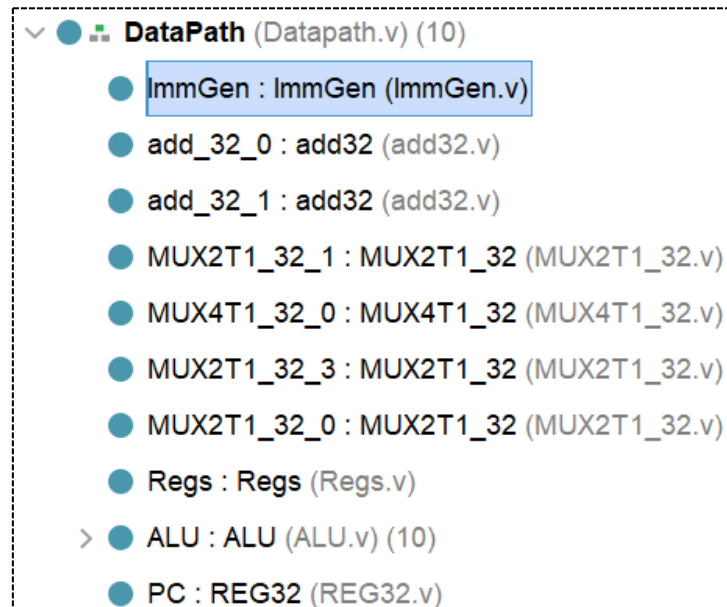
```

```

REG32 PC(
    .clk(clk),
    .rst(rst),
    .CE(1'b1),
    .D(PC_new),
    .Q(PC_out)
);
endmodule

```

(3) Lab04-1Datapath 工程结构:



(4) Datapath 仿真测试:

此处 Datapath 行为仿真，对 I、R、S、B、J 型指令都进行了仿真，同时对 add、sub、xor、beq、jal 等典型指令进行了测试（包含了部分寄存器的遍历）。

仿真代码如下:

```

`timescale 1ns / 1ps
module Datapath_tb();
    reg clk;
    reg rst;
    reg[31:0] inst_field;
    reg[31:0] Data_in;
    reg[2:0] ALU_operation;
    reg[1:0] ImmSel;
    reg[1:0] MemtoReg;
    reg ALUSrc_B;
    reg Jump;
    reg Branch;
    reg RegWrite;

```

```

wire[31:0] PC_out;
wire[31:0] Data_out;
wire[31:0] ALU_out;

DataPath U1(
    .clk(clk),
    .rst(rst),
    .inst_field(inst_field),
    .Data_in(Data_in),
    .ImmSel(ImmSel),
    .MemtoReg(MemtoReg),
    .ALU_operation(ALU_operation),
    .ALUSrc_B(ALUSrc_B),
    .Jump(Jump),
    .Branch(Branch),
    .RegWrite(RegWrite),
    .PC_out(PC_out),
    .Data_out(Data_out),
    .ALU_out(ALU_out)
);

always #5 clk = ~clk;
initial begin
    rst = 1;
    clk = 0;

    Branch = 0;
    Jump = 0;
    ALUSrc_B = 0;
    ALU_operation = 3'b010;
    MemtoReg = 2'b00;
    ImmSel = 2'b00;
    Data_in = 32'b00000000000000000000000000000000;
    RegWrite = 0; #4
    rst = 0;
    //I addi x3,x1,100
    inst_field = 32'b000001100100_00001_000_00011_0010011;
    ALUSrc_B = 1;
    ALU_operation = 3'b010;
    ImmSel = 2'b00;
    RegWrite = 1;
    MemtoReg = 2'b00;
    #10
    //R add x1,x2,x3

```

```

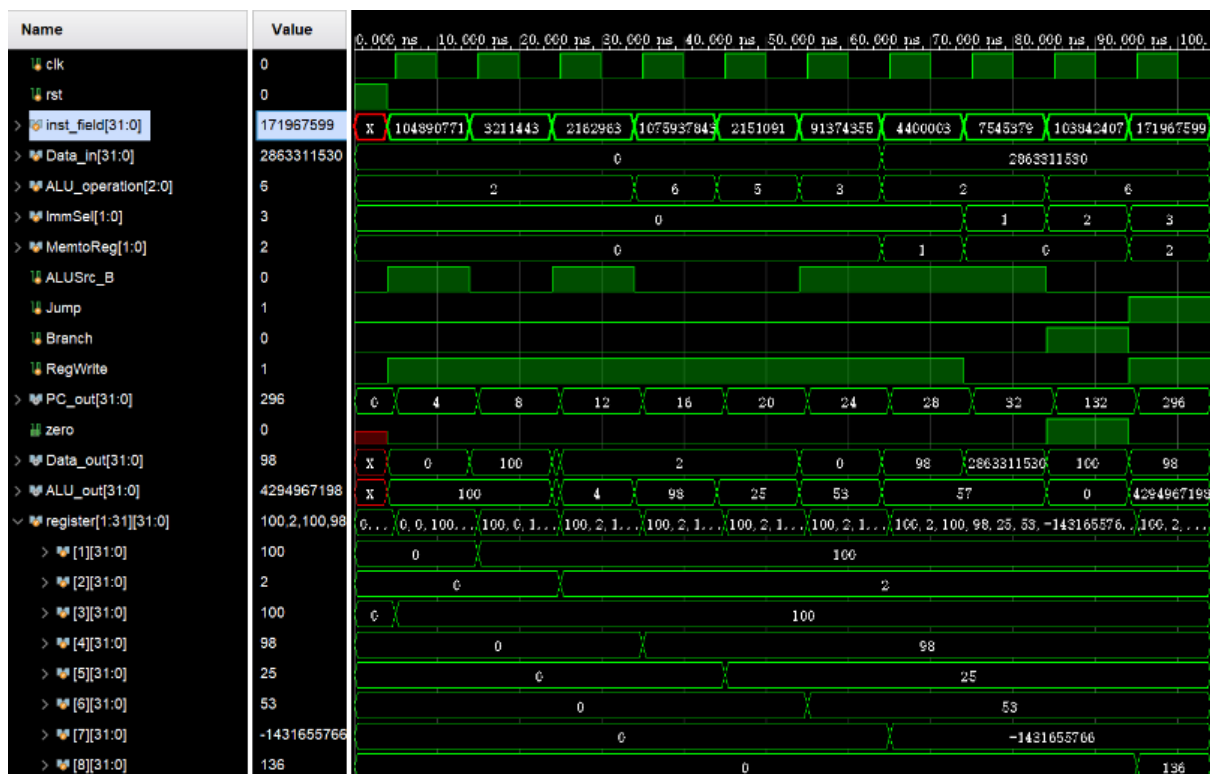
inst_field = 32'b00000000_00011_00010_000_00001_0110011;
ALU_operation = 3'b010;
RegWrite = 1;
ALUSrc_B = 0;
MemtoReg = 2'b00;    #10
//I addi x2,x2,2
inst_field = 32'b0000000000010_00010_000_00010_0010011;
ALUSrc_B = 1;
ALU_operation = 3'b010;
ImmSel = 2'b00;
RegWrite = 1;
MemtoReg = 2'b00;    #10
//R sub x4 x3 x2
inst_field = 32'b0100000_00010_00011_000_00100_0110011;
ALUSrc_B = 0;
ALU_operation = 3'b110;
RegWrite = 1;
MemtoReg = 2'b00;
#10
//R srl x5 x1 x2
inst_field = 32'b00000000_00010_00001_101_00101_0110011;
ALUSrc_B = 0;
ALU_operation = 3'b101;
RegWrite = 1;
MemtoReg = 2'b00;
#10
//I xori x6,x4,87
inst_field = 32'b000001010111_00100_100_00110_0010011;
ALUSrc_B = 1;
ALU_operation = 3'b011;
ImmSel = 2'b00;
RegWrite = 1;
MemtoReg = 2'b00;
#10
//I lw x7,4(x6)
inst_field = 32'b0000000000100_00110_010_00111_0000011;
ALUSrc_B = 1;
ALU_operation = 3'b010;
ImmSel = 2'b00;
RegWrite = 1;
Data_in = 32'b1010_1010_1010_1010_1010_1010_1010_1010;
MemtoReg = 2'b01;
#10
//S sw x7,4(x6)

```

```

inst_field = 32'b0000000_00111_00110_010_00100_0100011;
ALUSrc_B = 1;
ALU_operation = 3'b010;
ImmSel = 2'b01;
RegWrite = 0;
MemtoReg = 2'b00;
#10
//B beq x1,x3,100
inst_field = 32'b0000011_00011_00001_000_00100_1100111;
Branch = 1;
ALUSrc_B = 0;
ALU_operation = 3'b110;
ImmSel = 2'b10;
#10
// J jal x8 164
inst_field = 32'b0000101_00100_00000_000_01000_1101111;
Branch = 0;
Jump = 1;
RegWrite = 1;
MemtoReg = 2'b10;
ImmSel = 2'b11;
#10
$finish;
end
endmodule

```



四、实验结果分析

见上图仿真结果，加以分析（rst 初始化后寄存器值都为 0）：

- (1) `addi x3, x1, 100` (I 型) : $x3 = 100$
- (2) `add x1, x2, x3` (R 型) : $x1 = 0 + 100 = 100$
- (3) `addi x2, x2, 2` (I 型) : $x2 = 2$
- (4) `sub x4, x3, x2` (R 型) : $x4 = 100 - 2 = 98$
- (5) `srl x5, x1, x2` (R 型) : $x5 = 100 \gg 2 = 25$
- (6) `xori x6, x4, 87` (I 型) : $x6 = 1100010 \text{ xor } 1010111 = 0110101 = 53$
- (7) `lw x7, 4(x6)` (I 型) : 仅用来检测数据通路是否正确
- (8) `sw x7, 4(x6)` (S 型) : 仅用来检测数据通路是否正确
- (9) `beq x1, x3, 100` (B 型) : $x1 = x3 = 100$, 因此进行跳转

注意，此时 PC 地址为 32，那么下一次地址 PC_out 为 $32 + 100 = 132$.

Beq 跳转时 `zero = branch = 1`，使下一次 PC 地址选择 PC+100 而非 PC+4.

- (10) `jal x8, 164` (J 型) : $x8 = 132 + 4 = 136$, $PC_out = 132 + 164 = 296$

完成了 `jump` 的指令，并对 x8 进行了赋值。

综上所述，五类指令的正确性基本得到了验证，结果也符合预期，其中对于指令完备性和数据存储器模块（Data Memory）地测试会在 Lab4-03（所有指令均实现）后进行统一验证。

五、讨论与心得

(1) 思考题 1: 扩展下列指令, 数据通路将作如何修改:

R-Type: sra,sll,sltu; **I-Type:** srarai,sllr,slltiu ; **B-Type:** bne; **U-Type:** lui;

首先 R 型指令增加且超过了 8 个, 这意味着 3 位的 ALU_Control 不够分配, 需要将 Datapath 中的 ALU_control 修改成 4 位。

增加了 U 型指令 (大立即数), 2 位的 ImmSel 也不足控制, 需要增加到 3 位
此外, 添加 lui 后控制 Write data 的 MUX 需要添加位宽, 使得大立即数能够写回寄存器组。

```
MUX2T1_32 MUX2T1_32_1(  
    .I0(PC_inc),  
    .I1(PC_imm),  
    .s(Branch & ALU_zero),  
    .o(PC_1)  
);
```

同时添加 bne 判定 PC 跳转时, 如上图的 s 控制无法达到要求, 也需要更改。

(2) 思考题 2: 增加 I-Type 算术运算指令是否需要修改本章设计的数据通路:

在 R-Type 增加后的基础上不需要, 但是最终数据通路与本章设计的有所不同。因为 I-Type 指令还是基于 R-Type 的数据通路, 对于 ALU 的依赖, 就不可避免地导致 ALU_Control 需要增加到 4 位。

本次实验旨在用简单的逻辑模块组成 CPU 的重要部件 Datapath, 从底层出发可以发现其中的结构并没有想象中的高端 (也可能我们实现的比较低级)。

亲手搓一个简单的 CPU 还是蛮有趣的。